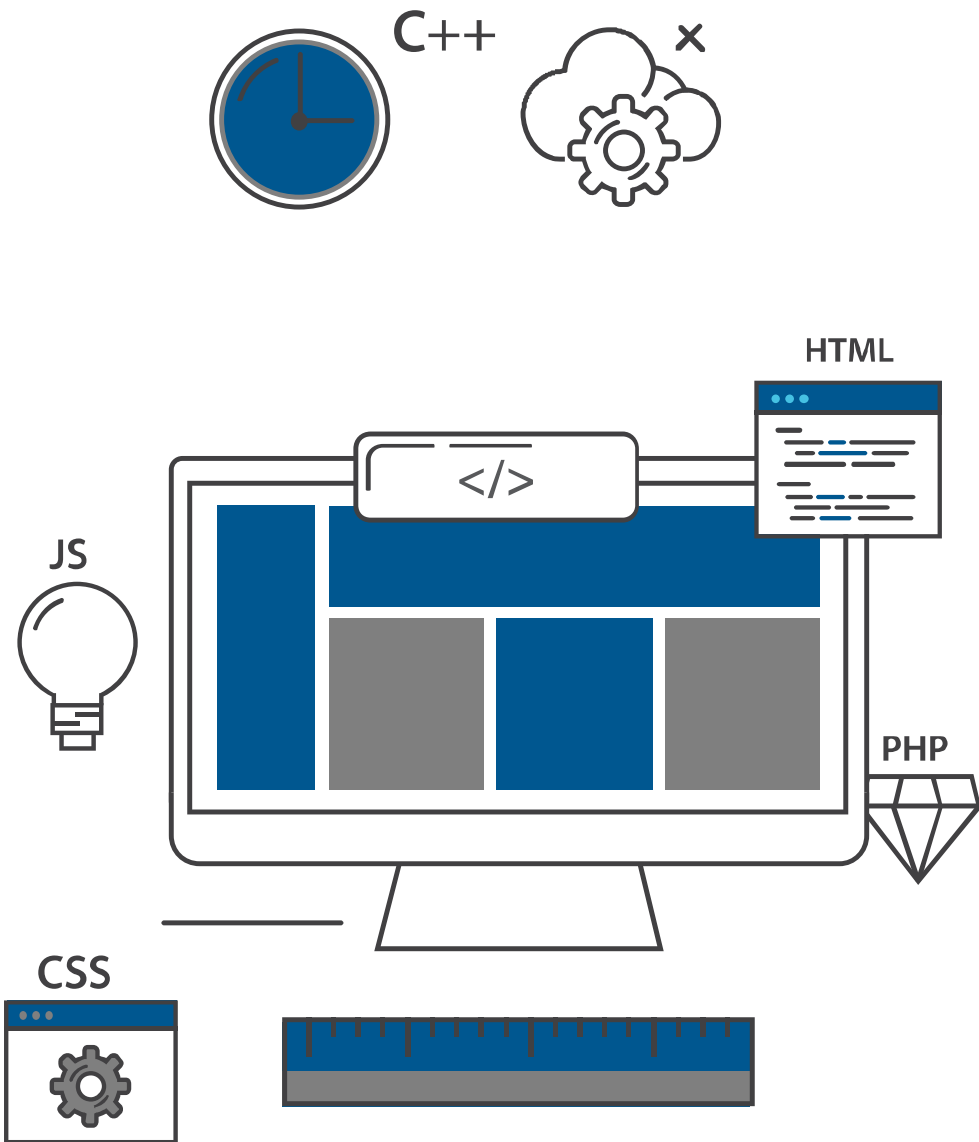
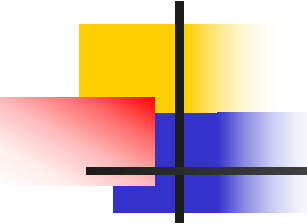


第六章 动态规划

李翔

<https://implus.github.io/>





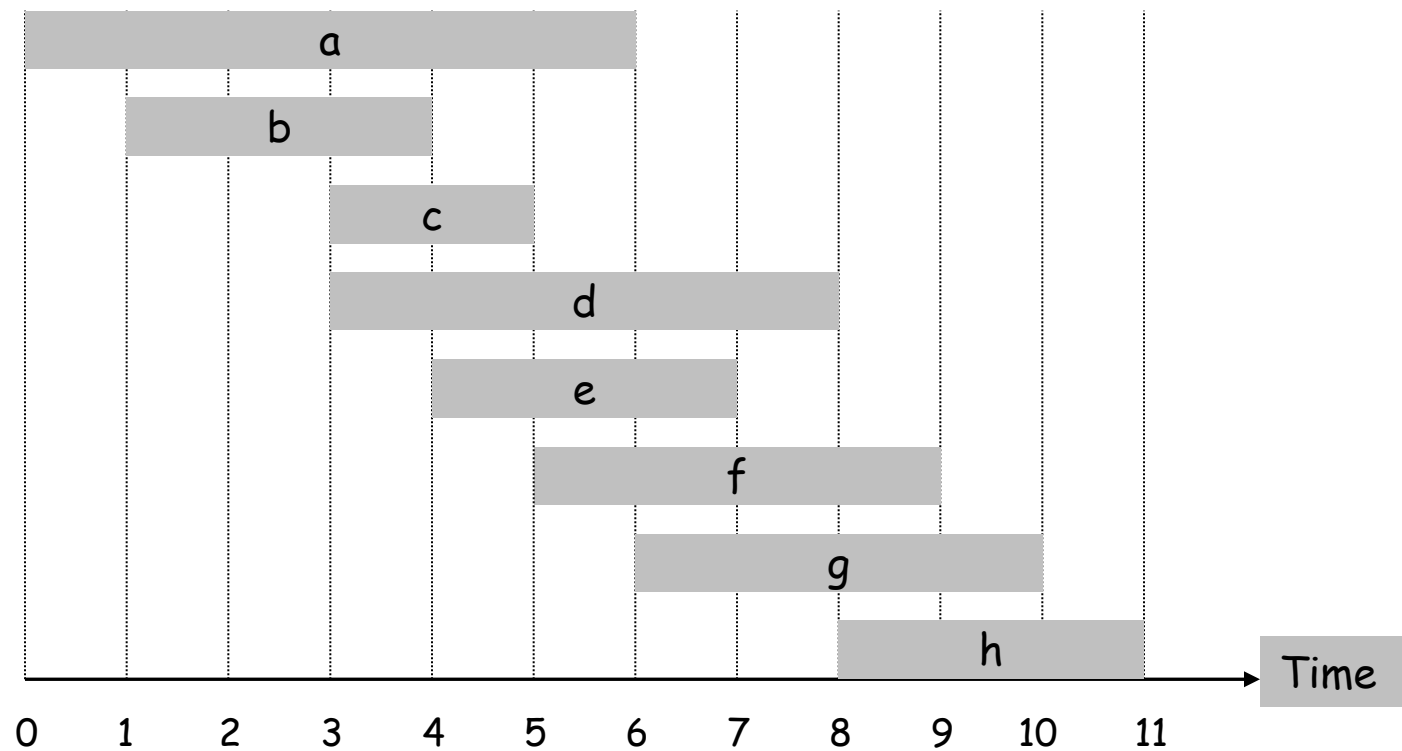
6.1 带权的区间调度

问题的提出

- 对于区间调度问题，不同区间有一样的权重，贪心算法可以得到最优解
- 如果不同区间有不同的权重：
 - 任务 j 在 s_j 时间开始， f_j 时刻结束，且其权重为 v_j .
 - 两个任务如果对应的时间区间不相交，称为**相容**.
 - 目标：寻找最大的不相容的区间子集，使得所选区间的**权重之和**最大。

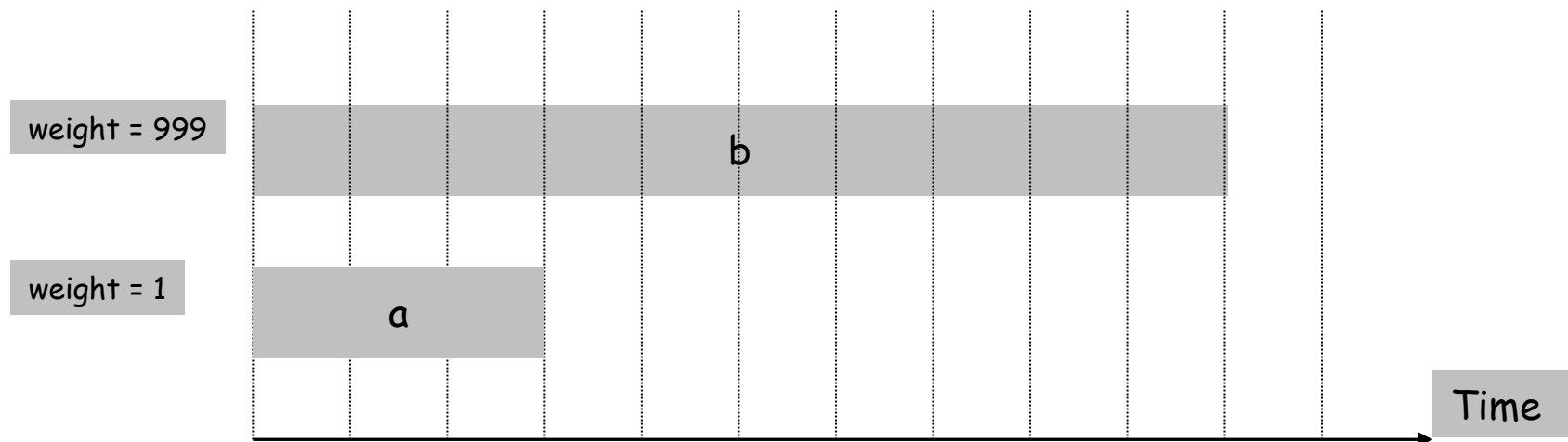
带权的区间调度

不同区间的影响是不一样的

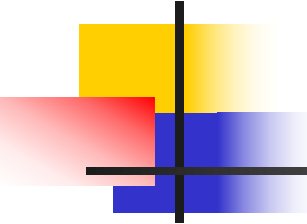


带权的区间调度

- 贪心算法情形下，区间的权重可以看成都是1
- 对于区间权重不同的情形，贪心算法就失效了



解决之道：更加灵活的调度策略



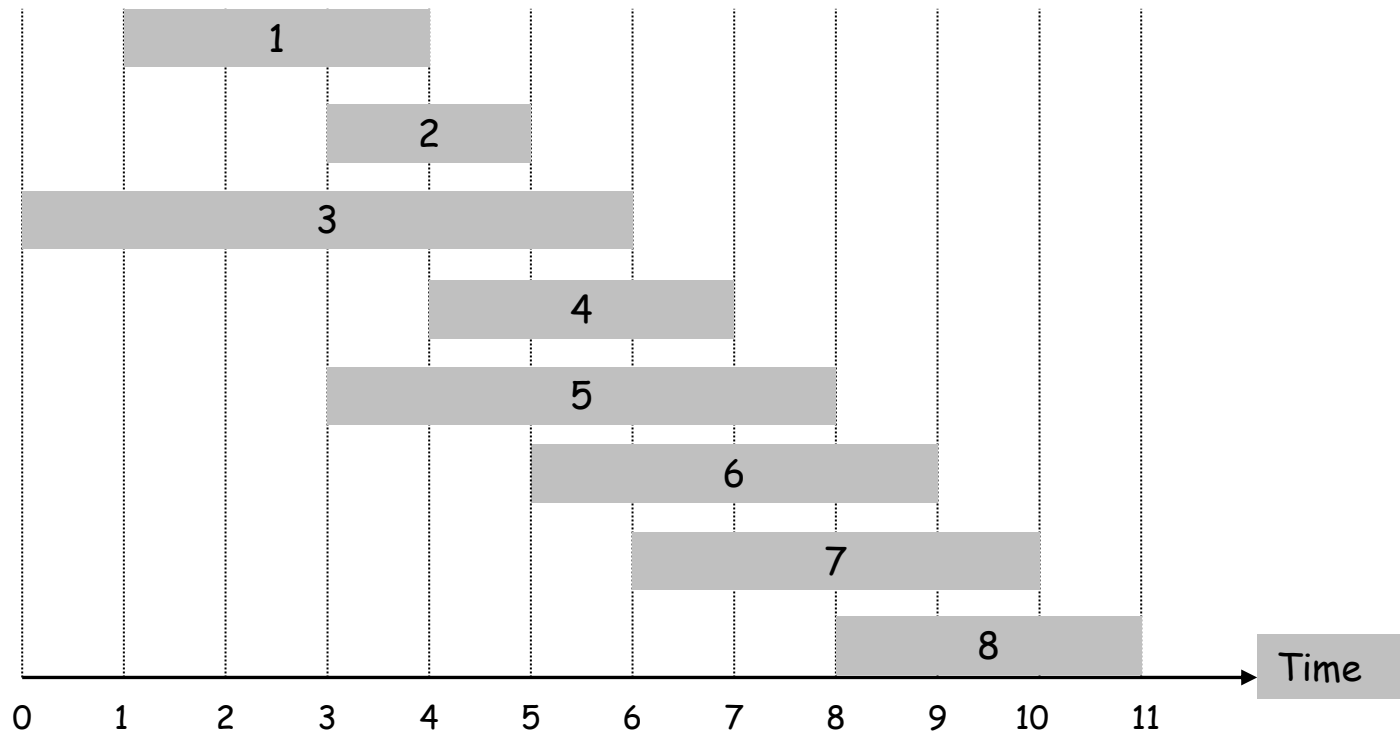
带权的区间调度

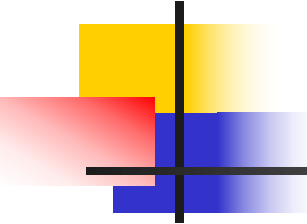
- 定义：任务需求按照结束时间排序： $f_1 \leq f_2 \leq \dots \leq f_n$.
- 定义：我们对区间j定义 $p(j)$ 为使得区间i与j不相交的最
大的标记 $i < j$, 也就是说, i是最右边的与j不冲突的区间。

带权的区间调度

- Ex: $p(8)$, $p(7)$, $p(2)$ =?

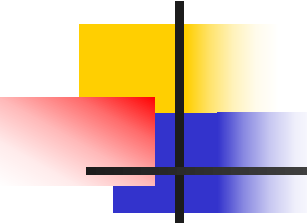
$$p(8) = 5, p(7) = 3, p(2) = 0$$





带权的区间调度

- 对于任意在1与n之间的j值,令 O_j 表示对于有需求 $\{1, \dots, j\}$ 所组成问题的最优解, 并且令 $OPT(j)$ 表示这个解的值。(从1考虑到j)
- 我们寻找的最优解就是 O_n , 具有值 $OPT(n)$.
- 下面将关注 $OPT(j)$ 的结构

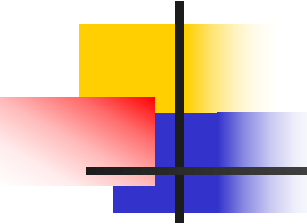


带权的区间调度

- 情形1: OPT 包含任务需求 j .
 - 不会包含不相容的任务需求 $\{ p(j) + 1, p(j) + 2, \dots, j - 1 \}$
 - 包含剩下的任务需求 $1, 2, \dots, p(j)$ 的最优解
- 情形2: OPT 不包含任务需求 j .
 - 一定包含任务需求 $1, 2, \dots, j-1$ 的最优解

定理 6.1

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max \{ v_j + OPT(p(j)), OPT(j-1) \} & \text{otherwise} \end{cases}$$

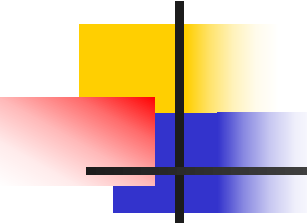


带权的区间调度

- 定理6.2 需求 j 属于集合 $\{1, 2, \dots, j\}$ 上的最优解当且仅当

$$v_j + OPT(p(j)) \geq OPT(j-1)$$

上面定理构成了动态规划求解基础的最重要的部分：用较小子问题的最优解来表达更大规模问题的最优解的一个递推的等式。



带权的区间调度

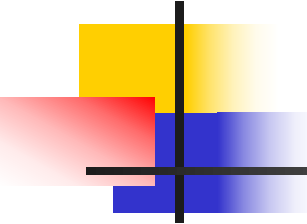
- 根据上面的性质，首先我们先写出如下的算法

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 

Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .

Compute  $p(1), p(2), \dots, p(n)$ 

Compute-Opt( $j$ )
{
    if ( $j = 0$ )
        return 0
    else
        return  $\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j-1))$ 
}
```



带权的区间调度

- 该算法的正确性:
- 定理6.3 对每个 $j=1,2,\dots,n$, $\text{Compute-Opt}(j)$ 正确地计算了 $\text{OPT}(j)$.

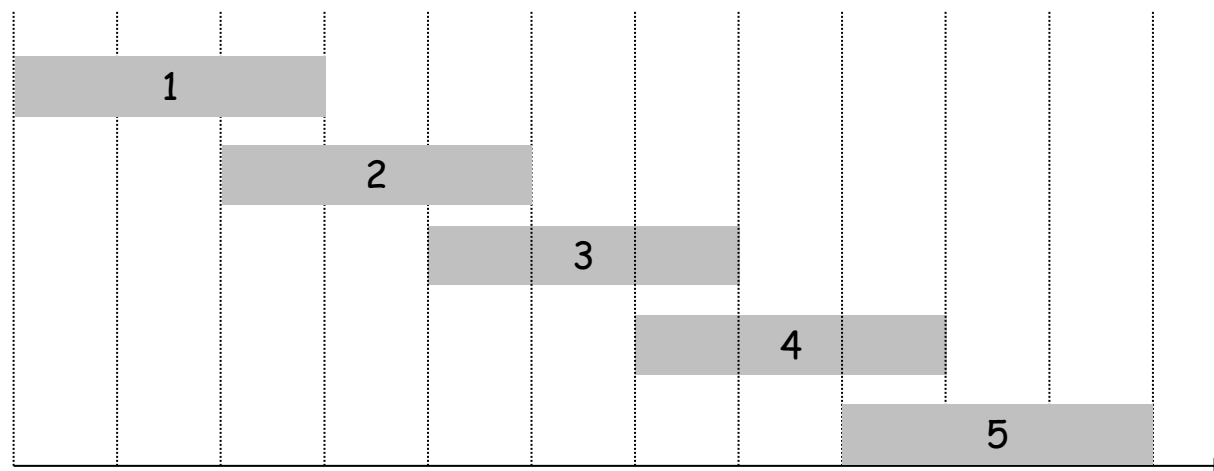
这个算法是不是一个**好**的多项式阶的算法呢?

最坏情形下这个算法将以**指数时间**运行!

带权的区间调度

- 观察 递归算法特别容易产生冗余的重复子问题计算, 从而导致了指数阶算法.

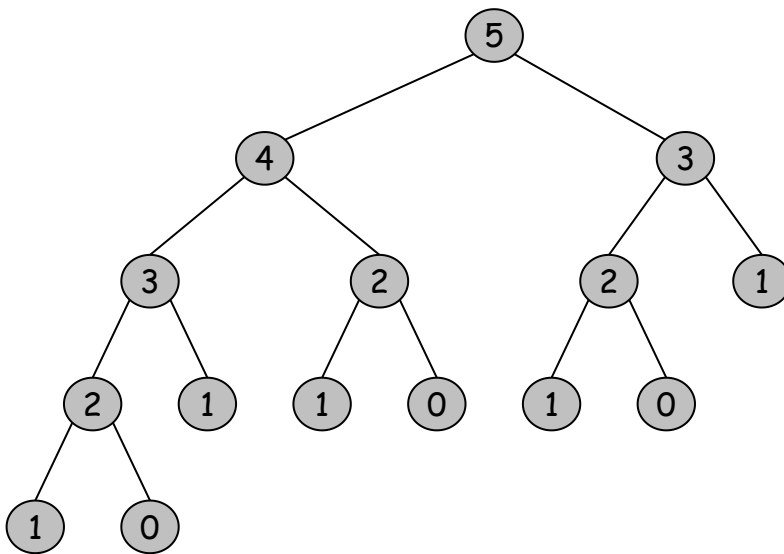
■ Ex.



$$p(1) = 0, p(j) = j-2$$

带权的区间调度

- 递归调用的次数增长就像Fibonacci 序列一样.

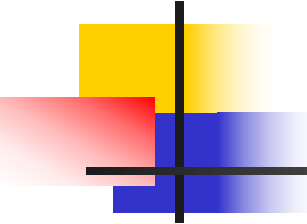


Fibonacci Sequence:

$$F(1)=F(2)=1,$$

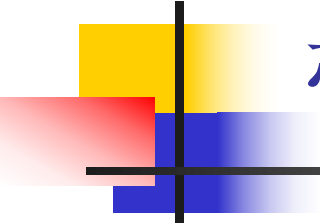
$$F(n)=F(n-1)+F(n-2)$$

$$F(n) = (1/\sqrt{5}) * \left\{ \left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right\}$$



带权的区间调度

- 递归的备忘录形式
- 为了避免上面的重复计算，我们把中间计算的结果存储起来，需要的时候先查找是否计算过
- 下面将用到一个数组 $M[0..n]$ 保存中间计算结果



带权的区间调度

Input: $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$

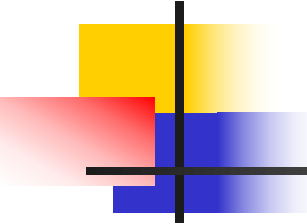
Sort jobs by finish times so that $f_1 \leq f_2 \leq \dots \leq f_n$.

Compute $p(1), p(2), \dots, p(n)$

```
for j = 1 to n
    M[j] = empty
M[0] = 0
```

← 全局数组

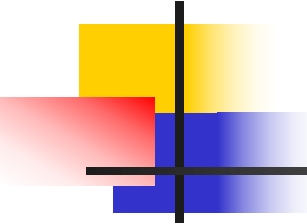
```
M-Compute-Opt(j) {
    if (M[j] is empty)
    {
        M[j] = max( $w_j + \text{M-Compute-Opt}(p(j))$ ,
                    $\text{M-Compute-Opt}(j-1)$ )
    }
    return M[j]
}
```



带权的区间调度

- 分析采用备忘录的算法
 - 按照结束时间排序: $O(n \log n)$.
 - 计算 $p(\cdot)$: 如果已对开始时间排序, 只需 $O(n)$

定理6.4 $M\text{-Compute-Opt}(n)$ 的运行时间是 $O(n)$.



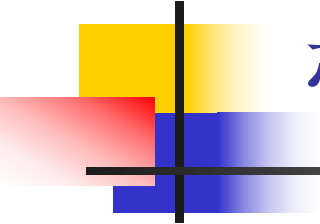
带权的区间调度

- 前面的动态规划算法只计算了一个最优解的值，如果要输出这组最优的区间的集合，如何处理？

利用存储的M[]数组进行后期处理

根据定理6.2，j属于一个区间集合 $\{1, \dots, j\}$ 的最优解当且仅当

$$v_j + OPT(p(j)) \geq OPT(j-1)$$

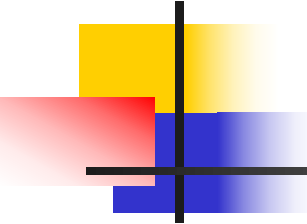


带权的区间调度

```
Run M-Compute-Opt(n)
Run Find-Solution(n)

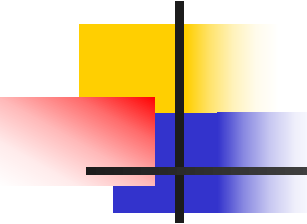
Find-Solution(j) {
    if (j = 0)
        output nothing
    else if ( $v_j + M[p(j)] > M[j-1]$ )
        print j
        Find-Solution(p(j))
    else
        Find-Solution(j-1)
}
```

算法的复杂度? $O(n)$



动态规划原理

- 对于带权区间调度问题，我们设计了一个多项式时间的算法：先设计一个（指数时间的）递归算法，然后通过备忘录将其转换成一个有效的递推算法，这个算法利用了全局数组 $M[]$ 来记录递归函数值。
- 提出另外一种解决思路：在子问题上迭代，而不是递归的计算解



动态规划原理

- 对于带权区间调度问题，我们可以不使用备忘录式的递归方法，而通过迭代算法直接计算M[]中的项。

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 
```

```
Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .
```

```
Compute  $p(1), p(2), \dots, p(n)$ 
```

```
Iterative-Compute-Opt
```

```
{
```

```
     $M[0] = 0$ 
```

```
    for  $j = 1$  to  $n$ 
```

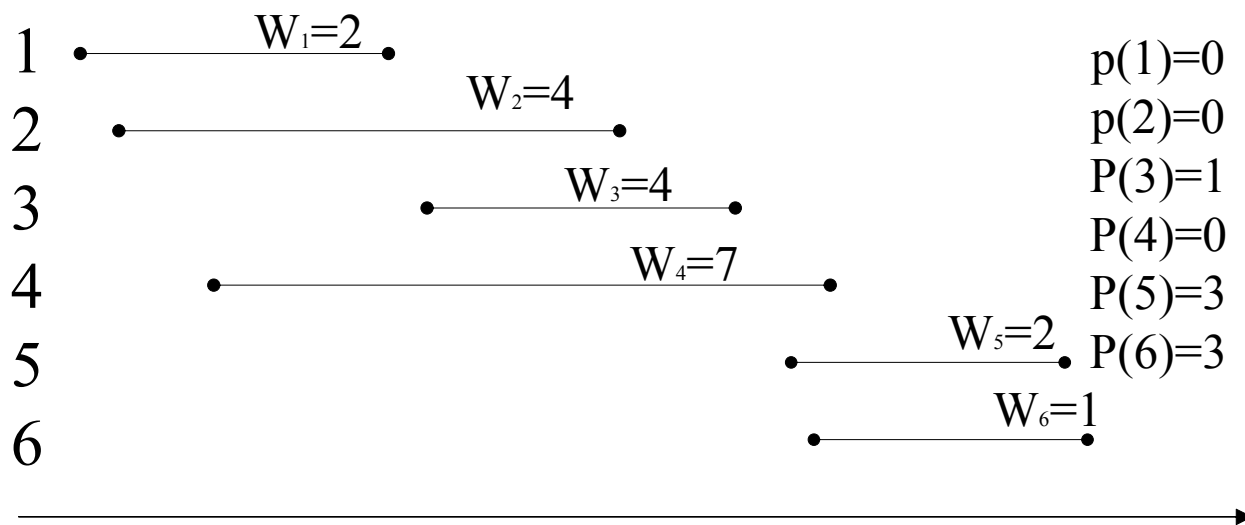
```
         $M[j] = \max(v_j + M[p(j)], M[j-1])$ 
```

```
}
```

自底向上

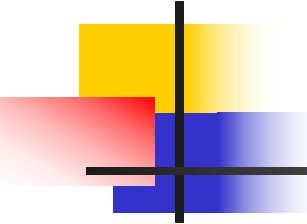
动态规划原理

- **Iterative-Compute-Opt**的运行时间是 $O(n)$ ，上面算法清楚的运行了 n 次迭代运算。



$p(1)=0$
 $p(2)=0$
 $P(3)=1$
 $P(4)=0$
 $P(5)=3$
 $P(6)=3$

$M = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \end{matrix}$						
0	2					
0	2	4				
0	2	4	6			
0	2	4	6	7		
0	2	4	6	7	8	
0	2	4	6	7	8	8



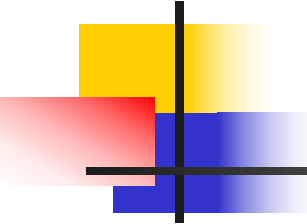
动态规划原理与分治策略

- 相似之处

- 通过子问题的解来求出全局问题的解

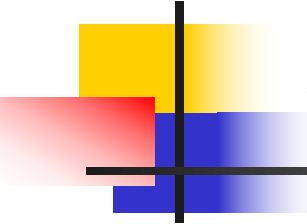
- 不同之处

- 子问题不独立
- 把子问题的解存在备忘录中



6.2 分段的最小二乘

- 问题的提出
- 绘制在一组二维数轴上的科学统计数据，试图用一条“最佳拟合”的线穿过这些数据
- 统计学与数值分析中的基本问题



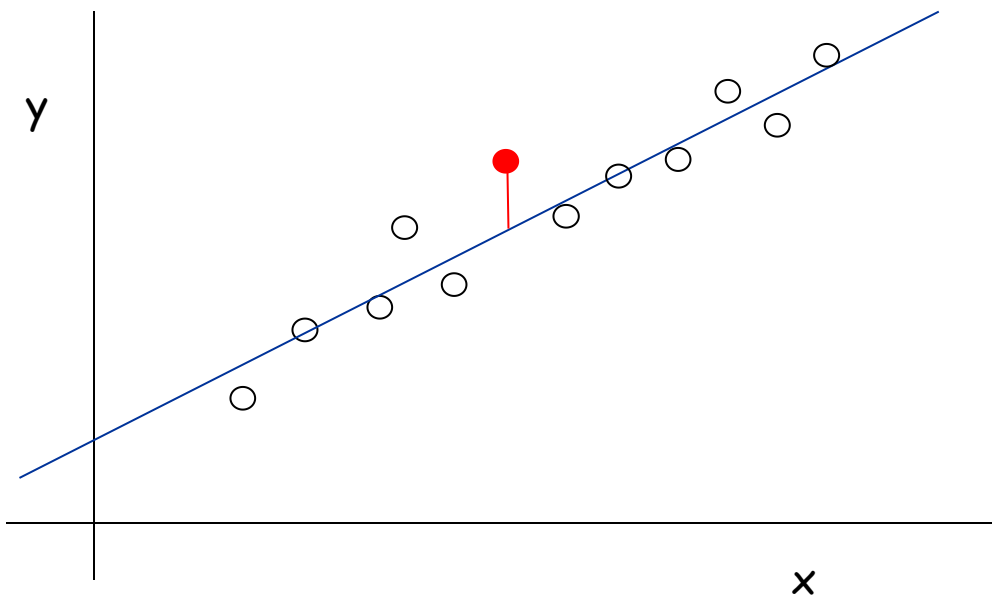
分段的最小二乘

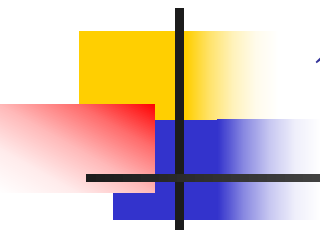
- 给定平面上的由 n 个点组成的集合 $P: (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$.
- 假设 $x_1 < x_2 < \dots < x_n$
- 给定由方程 $y=ax+b$ 定义的直线 L , 我们说 L 相对于 P 的误差是它对于 P 中点的“距离”的平方和;
- 自然的目标是寻找具有最小误差的直线

分段的最小二乘

误差

$$SSE = \sum_{i=1}^n (y_i - ax_i - b)^2$$



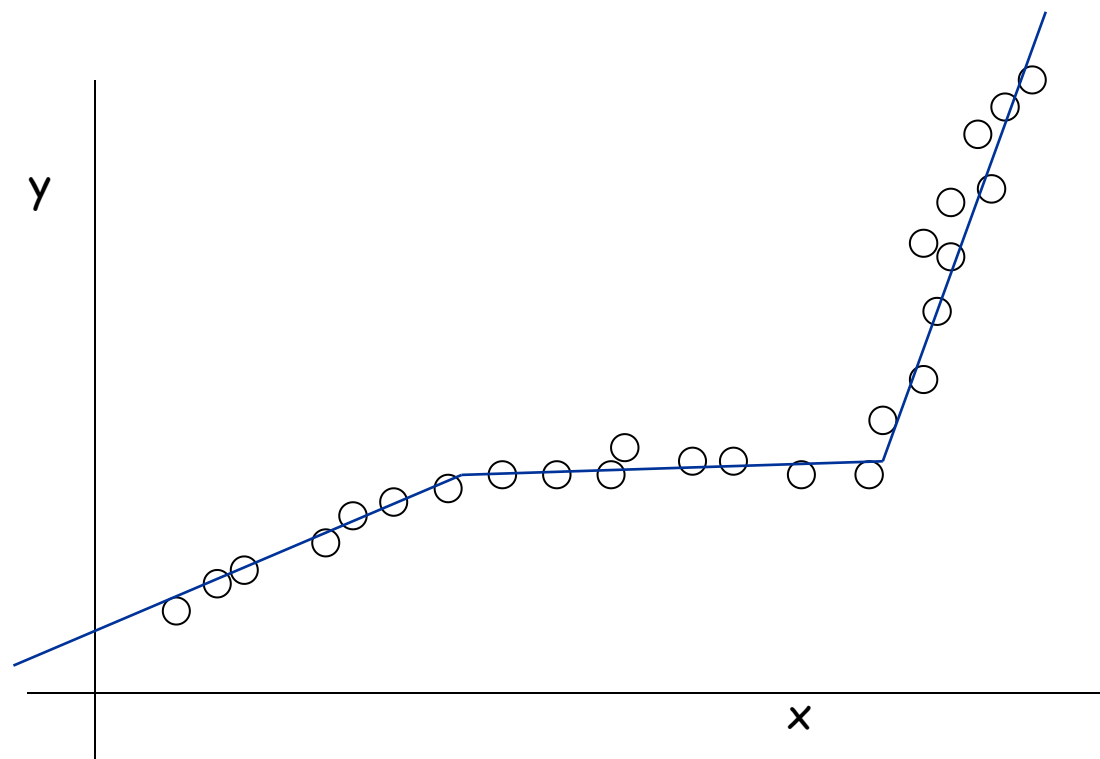


分段的最小二乘

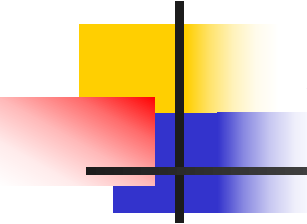
- 求极值的问题
- 通过微积分中的方法, 可以知道

$$a = \frac{n \sum_i x_i y_i - (\sum_i x_i) (\sum_i y_i)}{n \sum_i x_i^2 - (\sum_i x_i)^2}, \quad b = \frac{\sum_i y_i - a \sum_i x_i}{n}$$

分段的最小二乘

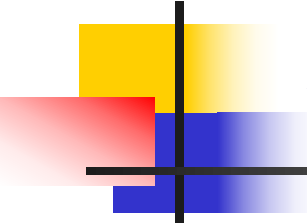


如何在拟合的**精确度**和使用最少的直线**数目**之间找到一个平衡？



分段的最小二乘

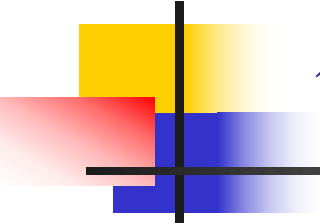
- 分段的最小二乘问题，问题形式化
- 给定一组点 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，其中 $x_1 < x_2 < \dots < x_n$
- 寻找一组直线使得：在所有直线上的平方误差之和 E ，直线的数目 L 之间找到一个好的平衡
 - 因此定义罚分度量函数： $E + cL$ ，其中 $c > 0$ 是某一个常数。



分段的最小二乘

- 采用动态规划的方法找到一个多项式时间的算法
- 定义.
 - $OPT(j)$ = 对于点 $p_1, p_{i+1}, \dots, p_j$ 的最优解
 - $e(i, j)$ = 关系到 $p_i, p_{i+1}, \dots, p_j$ 的任何直线的最小误差

递推式?

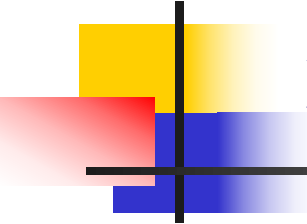


分段的最小二乘

- 定理6.7 对于 $p_1, \dots, p_j$ 的子问题,

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \min_{1 \leq i \leq j} \{ e(i, j) + c + OPT(i-1) \} & \text{otherwise} \end{cases}$$

现在设计这个算法中的困难部分已经克服：我们将给出一个自底向上的算法



算法

INPUT: $n, p_1, \dots, p_N, c$

Segmented-Least-Squares() {

$M[0] = 0$

for $j = 1$ to n

for $i = 1$ to j

compute the least square error e_{ij} for
 the segment $p_i, \dots, p_j$

for $j = 1$ to n

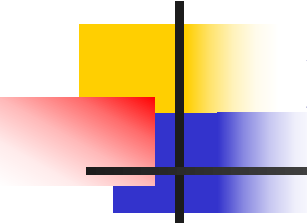
$M[j] = \min_{1 \leq i \leq j} (e_{ij} + c + M[i-1])$

return $M[n]$

}

复杂度分析

- 算法的前半部分，计算 $O(n^2)$ 个 $e(i, j)$ ，根据前面给出的公式计算每一对 $e(i, j)$ 需要 $O(n)$ 的复杂度；
后半部分，总的代价应该是 $O(n^2)$ ；
- 因此总的算法代价应该是 $O(n^3)$ ；如果预先计算了统计信息 $e(i, j)$ ，那么代价就是 $O(n^2)$ 。



算法

- 通过数组M向回寻找计算最优的划分

Find-Segments(j)

If $j=0$ then

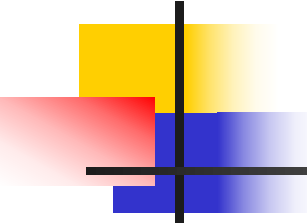
不用输出

Else

找一个使得 $e(i,j)+c+M[i-1]$ 最小的 i

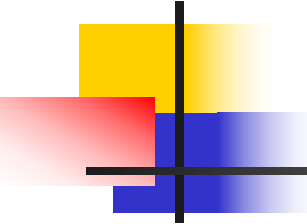
输出这一段 $\{p_i, \dots, p_j\}$ 作为直线段, 以及输出递归函数Find-Segments($i-1$)的结果

End if



子集和问题

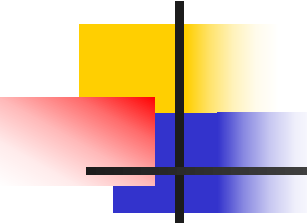
- 给定 n 个项 $\{1, \dots, n\}$, 每个项有一个给定的非负的权 w_i , 以及给定一个界 W . 我们想选择项的一个子集 S 使得 $\sum_{i \in S} w_i \leq W$, 并且在这个前提下使得 $\sum_{i \in S} w_i$ 达到最大。这个问题称为子集和问题。
- 这是更一般问题背包问题的特殊情形



背包问题

Knapsack Problem

- 用物品 $\{1, \dots, n\}$ 的子集装入一个容量 W 的背包使得它装的最满（或者装入的价值最大）。
- 每一个需求 i 有一个值 v_i 与一个权 w_i , 在总权不超过 W 的限制下，选择一个最大总值的子集。



子集和与背包

约束条件: $\sum_{i \in S} w_i \leq W$

要求, 最大化 $\sum_{i \in S} v_i$

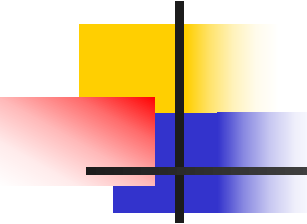
例子:

$W = 11$

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

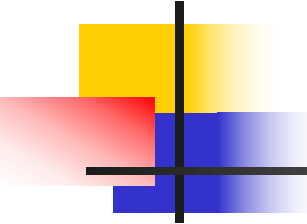
最大的价值? 选择{ 3, 4 }, 价值 40.

$\Rightarrow$ 贪心算法得不到最优解!



子集和与背包

- 动态规划方法:
- 定义 $OPT(i, w)$ 表示使用物品 $\{1, \dots, i\}$ 的子集且所允许的最大权是 w 的最优解的值。
(考虑到 $\{1, \dots, i\}$, 选取其中一个子集, 最大容量是 w)
- ✓ 情形1 OPT 没有选择 i ;
 OPT 选择子集 $\{1, 2, \dots, i-1\}$, 采用 w 的最优解
- ✓ 情形2 OPT 选择 i ;
 余下部分 OPT 选择子集 $\{1, 2, \dots, i-1\}$, 采用 $w-w_i$ 的最优解

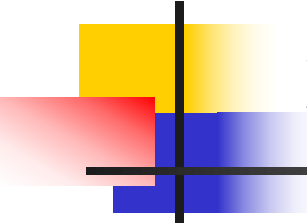


子集和与背包

- 据此马上我们就得到了递推式

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w - w_i) \} & \text{otherwise} \end{cases}$$

由此可见，增加一个变量重新进行动态规划，会变成更小的子问题，有助于设计动态规划算法。



算法

- 自底向上的算法
- 填满一个 $n \times W$ 的二维数组

```
Input:  $n, w_1, \dots, w_N, v_1, \dots, v_N$ 

for  $w = 0$  to  $W$ 
     $M[0, w] = 0$ 

for  $i = 1$  to  $n$ 
    for  $w = 1$  to  $W$ 
        if ( $w_i > w$ )
             $M[i, w] = M[i-1, w]$ 
        else
             $M[i, w] = \max \{M[i-1, w], v_i + M[i-1, w-w_i]\}$ 

return  $M[n, W]$ 
```

算法

		$W + 1$											
		0	1	2	3	4	5	6	7	8	9	10	11
$n + 1$	ϕ	0	0	0	0	0	0	0	0	0	0	0	0
	{ 1 }												
	{ 1, 2 }												
	{ 1, 2, 3 }												
	{ 1, 2, 3, 4 }												
	{ 1, 2, 3, 4, 5 }												

$W = 11$

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

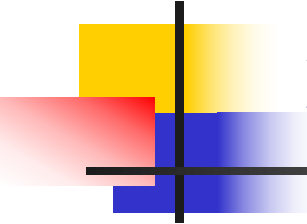
算法

		W + 1											
		0	1	2	3	4	5	6	7	8	9	10	11
n + 1	ϕ	0	0	0	0	0	0	0	0	0	0	0	0
	{ 1 }	0	1	1	1	1	1	1	1	1	1	1	1
	{ 1, 2 }	0	1	6	7	7	7	7	7	7	7	7	7
	{ 1, 2, 3 }	0	1	6	7	7	18	19	24	25	25	25	25
	{ 1, 2, 3, 4 }	0	1	6	7	7	18	22	24	28	29	29	40
	{ 1, 2, 3, 4, 5 }	0	1	6	7	7	18	22	28	29	34	35	40

OPT: { 4, 3 }
value = 22 + 18 = 40

W = 11

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7



算法

时间复杂度?

- 我们正在建立一个二位数组的表，并且我们在 $O(1)$ 时间内计算每个值 $M[i,w]$;于是得到
- 定理6.9 上面算法正确的计算这个问题的最优值，并且运行在 $\Theta(n W)$ 时间。

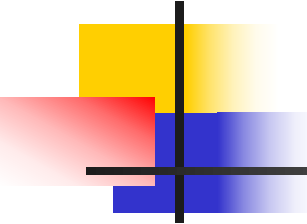
6.5 RNA二级结构

问题描述

- Watson, Crick提出双螺旋的DNA结构



- 单链RNA分子倾向于弯回去并且与自己构成碱基对；了解这种结构有助于了解分子行为的基础
- 配对的四种基本单位{ A, G, C, U }.

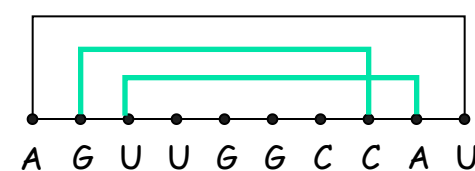
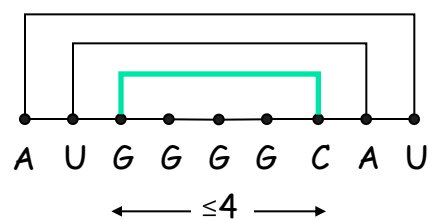
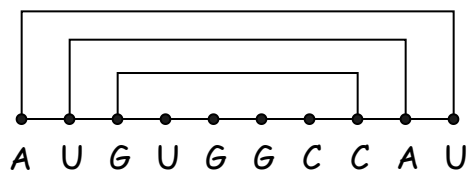
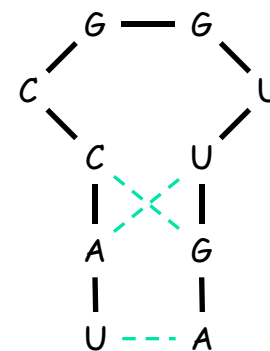
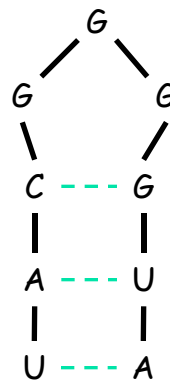
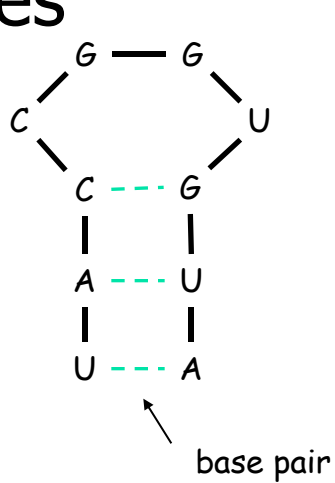


RNA二级结构

- 二级结构. 一些配对 $S = \{ (b_i, b_j) \}$ 满足:
 - [Watson-Crick.] S 是一个配对集合, 其中的每一个配对符合 Watson-Crick 配对规则: A-U, U-A, C-G, G-C.
 - [没有尖的转弯.] S 中每个对的两端被至少四个插入的碱基所分割. 即如果 $(b_i, b_j) \in S$, 那么 $i < j - 4$.
 - [不交叉.] 如果 (b_i, b_j) 与 (b_k, b_l) 是 S 中的两个对, 那么我们不能有 $i < k < j < l$.
 - 没有碱基出现在一个以上的对中。

RNA二级结构

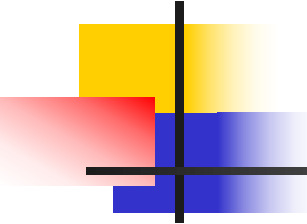
Examples



ok

尖的转弯

交叉



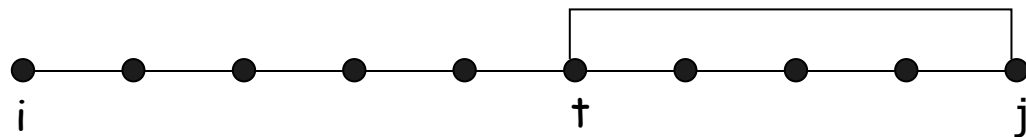
RNA二级结构

- 现在可以把基本的RNA二级结构预测问题描述的很客观:
- 对一个单螺旋RNA分子 $B = b_1b_2\dots b_n$, 确定具有最大碱基配对个数的二级结构S.

设计与分析算法

区间上的动态规划

- 定义 $\text{OPT}(i, j)$ 表示在 $b_i b_{i+1} \dots b_j$ 上的二级结构中碱基对的最大数目。
 - 情形 1. 如果 $i \geq j - 4$.
 - 根据没有尖的弯的规则, $\text{OPT}(i, j) = 0$.
 - 情形 2. 碱基 b_j 没有配对.
 - $\text{OPT}(i, j) = \text{OPT}(i, j-1)$
 - 情形 3. 碱基 b_j 与碱基 b_t 配对, 其中 $i \leq t < j - 4$.
- 根据不交叉的限制就把问题分成两个子问题
 - $\text{OPT}(i, j) = 1 + \max_t \{ \text{OPT}(i, t-1) + \text{OPT}(t+1, j-1) \}$

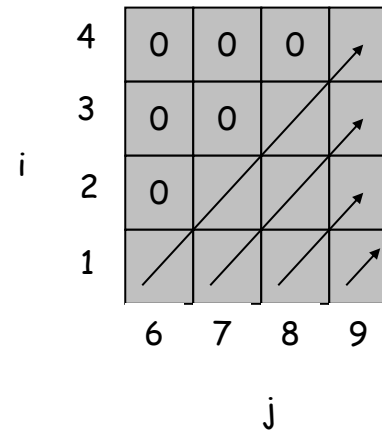


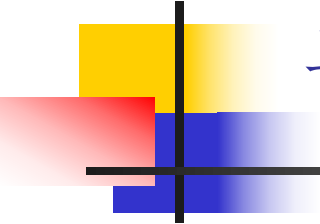
设计与分析算法

- 确认我们建立子问题解的适当顺序
- 按照区间长度增长的顺序建立解

```
RNA( $b_1, \dots, b_n$ ) {  
    for  $k = 5, 6, \dots, n-1$   
        for  $i = 1, 2, \dots, n-k$   
             $j = i + k$   
            Compute  $M[i, j]$   
  
    return  $M[1, n]$   
}
```

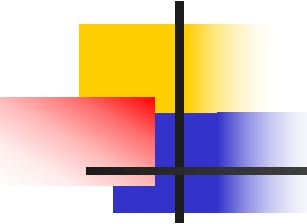
using recurrence





设计与分析算法

- 算法时间复杂度
- 存在 $O(n^2)$ 个子问题;每次求值用 $O(n)$ 时间
- 总运行时间是 $O(n^3)$



动态规划方法小结

- 动态规划技术
 - 两种情况里面挑选最优解：带权的区间调度.
 - 多种情况里面挑选最优解：分段的最小二乘.
 - 增加一个变量：背包问题.
 - 在区间上的动态规划：RNA 二级结构.
- 自顶向下(递归) vs 自底向上(循环)：不同的思路

序列比对

- 编辑距离 [Levenshtein 1966, Needleman-Wunsch 1970]
 - 空隙罚分 δ ; 不匹配罚分 α_{pq} .
 - 总的罚分 = 空隙罚分和不匹配罚分之和.

C T G A C C T A C C T

C C T G A C T A C A T

$$\alpha_{TC} + \alpha_{GT} + \alpha_{AG} + 2\alpha_{CA}$$

- C T G A C C T A C C T

C C T G A C - T A C A T

$$2\delta + \alpha_{CA}$$

序列比对

比对罚分

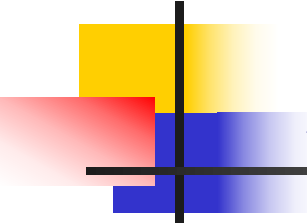
$$\text{cost}(M) = \sum_{\substack{(x_i, y_j) \in M \\ \text{mismatch}}} a_{x_i y_j} + \sum_{i: x_i \text{ unmatched}} \delta + \sum_{j: y_j \text{ unmatched}} \delta$$

Ex: CTACCG **vs.** TACATG.

$$M = x_2-y_1, x_3-y_2, x_4-y_3, x_5-y_4, x_6-y_6.$$

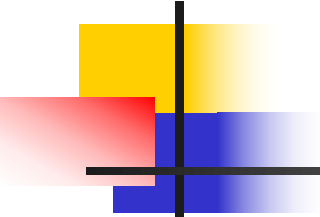
x ₁	x ₂	x ₃	x ₄	x ₅		x ₆
C	T	A	C	C	-	G

-	T	A	C	A	T	G
	y ₁	y ₂	y ₃	y ₄	y ₅	y ₆



序列比对

- 定理6.15 在一个最优比对 M 中, 至少下述情况之一为真
 - i. (m, n) 在 M 中;
 - ii. X 的第 m 个位置没有被匹配;
 - iii. Y 的第 n 个位置没有被匹配.



序列比对

- 定义 $\text{OPT}(i, j) = x_1 x_2 \dots x_i$ 与 $y_1 y_2 \dots y_j$ 比对的最小罚分
 - Case 1: $x_i - y_j$ 在 OPT 中
 - $x_i - y_j$ 不匹配罚分 + $x_1 x_2 \dots x_{i-1}$ 和 $y_1 y_2 \dots y_{j-1}$ 最小比对罚分
 - Case 2a: OPT 中 x_i 没有匹配
 - x_i 处间隙罚分 + $x_1 x_2 \dots x_{i-1}$ 和 $y_1 y_2 \dots y_{j-1} y_j$ 最小比对罚分
 - Case 2b: OPT 中 y_j 没有匹配
 - y_j 处间隙罚分 + $x_1 x_2 \dots x_{i-1} x_i$ 和 $y_1 y_2 \dots y_{j-1}$ 最小比对罚分

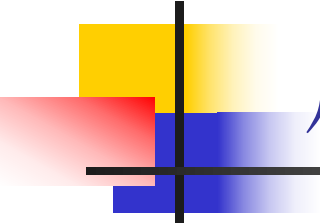
序列比对

- 所以递归式应该是

$$OPT(i, j) = \begin{cases} j\delta & \text{if } i = 0 \\ \min \begin{cases} a_{x_i y_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases} & \text{otherwise} \\ i\delta & \text{if } j = 0 \end{cases}$$

定理6.16 对于 $i, j \geq 1$, $OPT(i, j) = \min \begin{cases} a_{x_i y_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases}$

此外, (i, j) 在最优比对中, 当且仅当达到上面最小值。



序列比对算法

```
Sequence-Alignment(m, n,  $x_1x_2\dots x_m$ ,  $y_1y_2\dots y_n$ ,  $\delta$ ,  $a$ ) {  
    for i = 0 to m  
        M[0, i] =  $i\delta$   
    for j = 0 to n  
        M[j, 0] =  $j\delta$   
  
    for i = 1 to m  
        for j = 1 to n  
            M[i, j] = min( $a[x_i, y_j] + M[i-1, j-1]$ ,  
                           $\delta + M[i-1, j]$ ,  
                           $\delta + M[i, j-1]$ )  
  
    return M[m, n]  
}
```

时间，空间复杂度？

$\Theta(mn)$

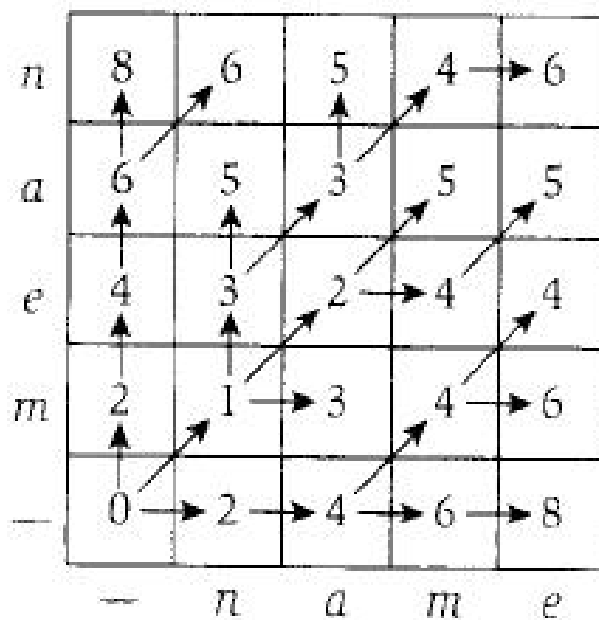
分析算法

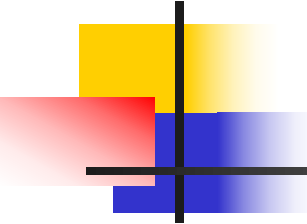
- 建立一个二维方格图 G_{xy} , 用来表示状态之间的转换。
- 例子: 间隙罚分 $\delta = 2$, 元音与不同元音配对, 辅音与不同辅音配对不匹配罚分1; 元音与辅音配对罚分3.

n					
a					
e					
m					
—					
	—	n	a	m	e

分析算法

- 建立一个二维方格图 G_{xy} , 用来表示状态之间的转换。
- 例子: 间隙罚分 $\delta = 2$, 元音与不同元音配对, 辅音与不同辅音配对不匹配罚分1; 元音与辅音配对罚分3.





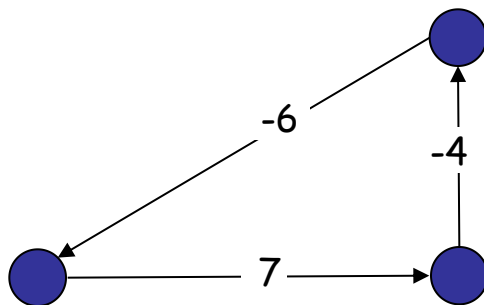
6.8 图中的最短路径

- 问题描述
- 令 $G = (V, E)$ 是一个有向图，假定每条边 (i, j) 有一个相关的权 C_{ij} ，表示从结点 i 直接到结点 j 的费用。
- 这里费用可能为负：比如，结点可能是表示在一个金融背景中的代理，在交易中我们从代理 i 买入，然后卖给代理 j 。

图中的最短路径

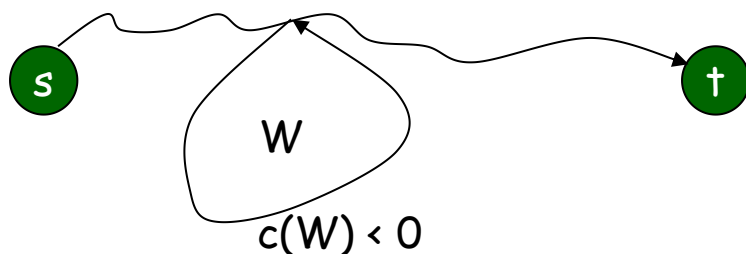
- 实际情形中，一个负圈与金融问题中一个有利可图的交易序列相对应，负圈可以看成一个好的盈利机会。

■ 负圈

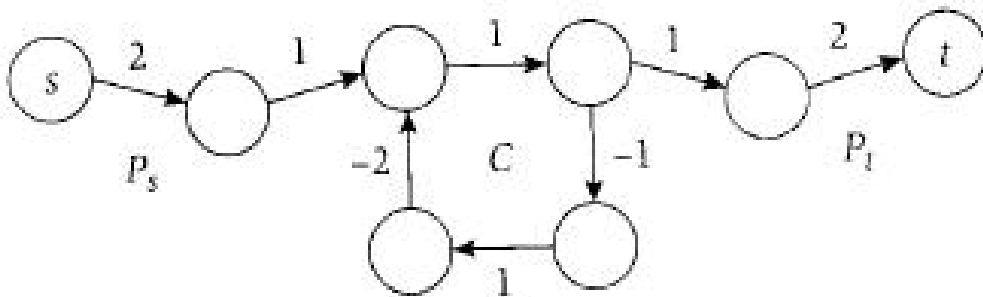


图中的最短路径

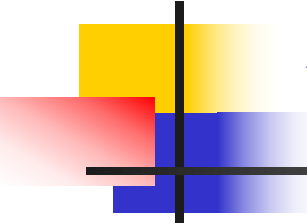
- 如果存在负圈，**s-t**中不存在“最短”路径



在这里，没有负圈的假定下考虑最小费用**s-t**路径才有意义

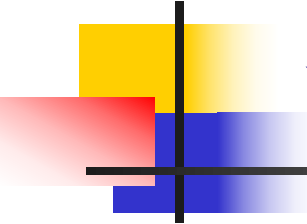


可以找到任意负费用!



设计与分析算法

- 动态规划方法
- Bellman-Ford最短路径算法
- 定理6.22 如果 G 没有负圈，那么存在一条从 s 到 t 的简单的最短路径(没有重复结点)，因此它至多有 $n-1$ 条边。



设计与分析算法

- $OPT(i, v)$ 表示至多使用*i*条边的v-t最短路径的最小费用, 我们目的是计算s到t的最短路径: $OPT(n-1, s)$
- 多重选择, $OPT(i, v)$
 - i. 如果路径P至多用*i-1*条边, 那么 $OPT(i, v) = OPT(i-1, v)$
 - ii. 如果路径P用*i*条边, 并且第一条边是(*v*, *w*), 那么余下的是w-t路径至多使用*i-1*条边:

$$OPT(i, v) = c_{vw} + OPT(i-1, w)$$

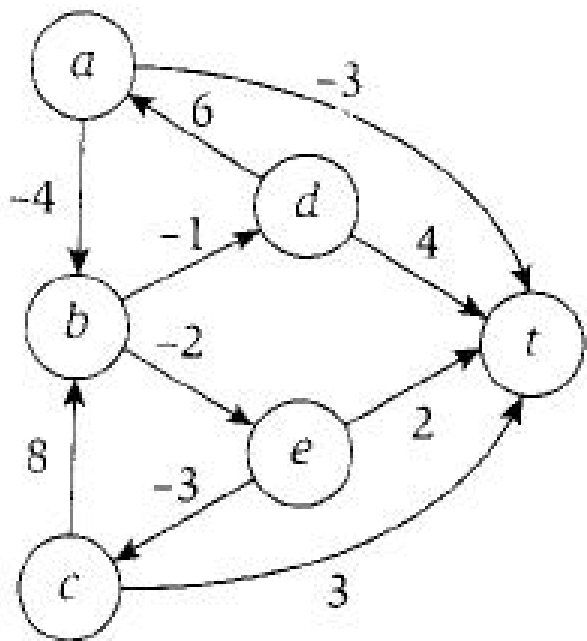
定理6.23

$$OPT(i, v) = \begin{cases} 0 & \text{if } i = 0 \\ \min \left\{ OPT(i-1, v), \min_{(v, w) \in E} \{ OPT(i-1, w) + c_{vw} \} \right\} & \text{otherwise} \end{cases}$$

根据前面的结论，如果图中没有负圈，那么
 $OPT(n-1, v)$ 就是最短的v-t路径长度。

设计与分析算法

■ 例子: Shortest-Path 算法



	0	1	2	3	4	5
t	0	0	0	0	0	0
a	∞	-3	-3	-4	-6	-6
b	∞	∞	0	-2	-2	-2
c	∞	3	3	3	3	3
d	∞	4	3	3	2	0
e	∞	2	0	0	0	0

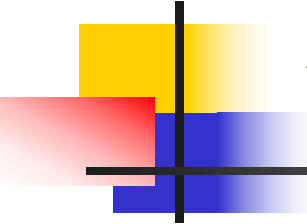
存储需求的改进

- 我们不对每个值*i*记录 $M[i,v]$,而是对每个结点*v*更新一个值 $M[v]$, 即我们**至今已经找到的从*v*到*t*的最短路径的长度**。

$$M[v] = \min(M[v], \min_{w \in V} (c_{vw} + M[w]))$$

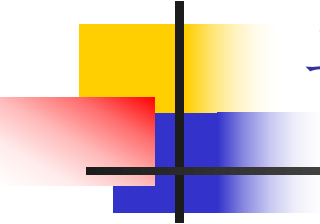
找到最短路径&存在性

- 为了帮助复原这条最短路径，记录每个结点通向终点 t 的路径上它后面的第一个结点；只要距离 $M[v]$ 更新，我们就更新这个值($first[v]$)。



设计与分析算法

- 若 G 没有负圈，那么指针图没有圈，而且终点是 t ，算法结束的时候就得到 v 到 t 的一条最短路径。
- 命题6.28 假设 G 没有负圈，考虑算法终止时的指针图 P . 对每个结点 v , 在 P 中从 v 到 t 的路径是 G 中的一条最短 v - t 路径。
- 算法终止信号：某一次迭代中所有的 $M[v]$ 值都保持不变。



设计与分析算法

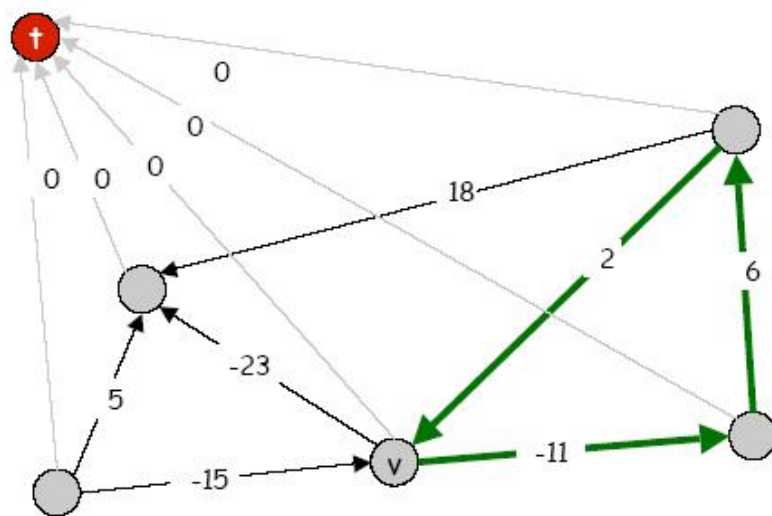
```
Push-Based-Shortest-Path( $G, s, t$ ) {  
    foreach node  $v \in V$  {  
         $M[v] \leftarrow \infty$   
         $\text{successor}[v] \leftarrow \phi$   
    }  
  
     $M[t] = 0$   
    for  $i = 1$  to  $n-1$  {  
        foreach node  $w \in V$  {  
            if ( $M[w]$  has been updated in previous iteration) {  
                foreach node  $v$  such that  $(v, w) \in E$  {  
                    if ( $M[v] > M[w] + c_{vw}$ ) {  
                         $M[v] \leftarrow M[w] + c_{vw}$   
                         $\text{successor}[v] \leftarrow w$   
                    }  
                }  
            }  
        }  
        If no  $M[w]$  value changed in iteration  $i$ , stop.  
    }  
}
```

* 图中的负圈

- 如何确定图中是否包含负圈？

G 有 n 个结点且 $OPT(n,v) \neq OPT(n-1,v)$

- 如何在包含负圈的图中找到负圈？



$O(mn)$